Correctness Boundaries of Typed Plans for Natural-Language Robot Fleet Control

Artem Tuzov[1*], Dmitrii Vizitei[1], Evgenii Shlomov[1], Senya Belykh[1], Anastasia Malakhova[1], Alexander Maloletov[1]

[1]Innopolis University, Innopolis, Russia.

*Corresponding author(s). E-mail(s): a.tuzov@innopolis.university;
Contributing authors: d.vizitei@innopolis.university;
e.shlomov@innopolis.university; a.malakhova@innopolis.university;
a.maloletov@innopolis.ru;

Abstract

Typed plans can separate language-level task interpretation from robot execution, but structural validity does not establish semantic correctness, geometric feasibility, or mission completion. We examine these boundaries through a three-level ROS 2 implementation deployed with twenty simulated mobile robots. The model emits recursive fleet plans; deterministic checks validate or transform selected actions; classical components coordinate and execute them. An offline benchmark of 44 cases with five repeats gives 70.0–87.3% structural pass rates for local models. The reported main series contains 18 runs completing their task-evaluator conditions on two mission specifications and one map. Here, a carry task means satisfying a dwell condition at a pickup location and then at a drop-off location, not physically transporting an object. Duplicate-carrier assignments occur in 5 of 13 language plans. A separate, uncontrolled M3 comparison reports 8/9 tasks before and 9/9 after destination spreading; its non-randomized design precludes pooling it with the main series. A formation failure and qualitative development observations expose gaps in geometric checks and completion reports. We organize these observations by check coverage and distinguish implemented mechanisms from evaluated effects. The evidence supports a bounded architectural case study, not a general reliability gain, a safety certificate, or a demonstration of twenty concurrently active robots.

Keywords: Multi-robot systems, Language models, Fleet coordination, Typed plans, Deterministic guards, ROS 2

1 Introduction

Commanding a fleet of mobile robots that share one workspace is a long-standing problem, and the classical decomposition separates fleet-level decisions — which robot goes where, in what order — from robot-level execution. Large language models make a natural-language front-end to that decomposition newly attractive [1, 2]: a model can reason over compact semantic primitives instead of producing control commands.

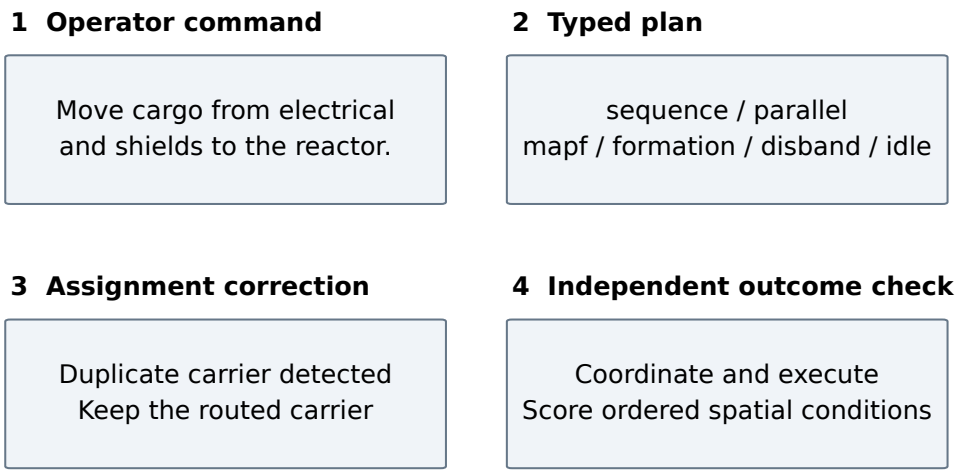


Fig. 1 Illustrative duplicate-carrier correction at the language–coordination boundary. The typed plan contains an extra carrier; a task-specific guard removes it before execution. Duplicate-carrier assignments occurred in 5 of 13 language plans. The illustration is not the shared-drop-off before/after experiment.

The difficulty is that a well-formed language response need not describe an executable fleet action. It may refer to unavailable robots, assign conflicting goals, or request a formation whose geometry is incompatible with a corridor. Direct MAPF generation also remains difficult in obstacle-rich environments [3]. We therefore separate task interpretation from spatial planning and ask which errors can be handled at their interface.

Hierarchical control, formal intermediate representations, and deterministic validation of language outputs are established approaches. Our contribution is an empirical characterization of their correctness boundaries in a specific ROS 2 fleet implementation, rather than a new general hierarchy or a claim that guards improve reliability across environments.

We ask which structural errors occur in generated fleet plans under the tested model and serving configurations, addressing RQ1. To understand what happens after generation, RQ2 asks which observed assignment and spatial conflicts the implemented guards handle and which they leave unchecked. Finally, RQ3 asks why a component

can report successful execution without fulfilling the mission's task-evaluator conditions. These questions follow a plan from generation through validation to its measured outcome.

We distinguish plan representation, intended task meaning, spatial executability, and evaluator completion. The contribution combines an explicit description of the interface and its check coverage with recorded carry and formation cases and qualitative development observations. These sources have different denominators and provenance; they are not independent replications of one guard-effect experiment. Figure 1 illustrates a duplicate-carrier correction. The shared-drop-off comparison is a separate case in Section 8.1.

2 Related Work

An instruction to a robot fleet leaves two different problems to solve: deciding what the robots should do and finding motions that realize those decisions. Classical multi-agent path finding addresses the latter once starts, goals and movement rules have been specified. Conflict-Based Search [4] resolves path conflicts through constraints, while prioritized search [5] and large-neighborhood repair [6] offer alternative ways to coordinate paths. The resulting paths coordinate motion under the specified rules, but do not determine whether the supplied assignments and goals express a user's request. A language interface must establish that connection before path planning can serve the intended task.

Grounding language in executable skills is therefore central to early LLM-based robot planners. SayCan [1] combines a language model's estimate of a skill's usefulness with an affordance estimate of whether the robot can execute it in the current state. This connects task interpretation to available behavior without asking the language model to generate motor commands. Code as Policies [2] gives the model a more expressive interface: it composes perception and control APIs into programs, including control flow and numerical operations. The distinction matters for fleet control because choosing an available action and assembling several actions into a coherent mission require different checks.

Execution can depart from a plan even when its actions are available. Inner Monologue [7] feeds observations and success reports back into language reasoning, allowing subsequent decisions to account for execution. ProgPrompt [8] incorporates this concern into program-like plans through assertions and recovery steps. In its simulated evaluation, assertions are assessed by an LLM using environment-state information; they are not the same mechanism as deterministic runtime guards. Both approaches make feedback part of planning. They also motivate a question that is important in our implementation: what does the reported success establish, and is that sufficient for the original task?

For a robot team, grounding individual actions must be combined with allocation and coordination [9]. SMART-LLM [10] uses prompted task decomposition, coalition formation and allocation to construct a team plan. This makes robot selection part of language-level planning rather than an input assumed to be correct. RoCo [11] instead uses dialogue between robot-specific language agents to develop a joint plan, which is

checked before a centralized multi-arm planner generates motion. These designs place coordination decisions at different points in the pipeline, but both require a connection between the proposed team behavior and the capabilities of its executor. Our duplicate-carrier and shared-destination cases examine that connection in a mobile fleet.

Formal planning provides another way to make those decisions explicit. LaMMA-P [12] combines language reasoning with PDDL-based heuristic planning for heterogeneous teams, so language interpretation can be followed by search over a planning model. HiP-CL [13] further separates translation from problem construction: instructions first become a controlled language, from which a planning problem and spatial and resource constraints are constructed. This separation clarifies where constraints enter the system. Its relevance here is that a planner can reason about constraints represented in its model; whether that model includes the geometry needed for a requested formation is a separate implementation question.

A narrower execution interface can also reject unsupported outputs before motion begins. CommandSwarm [14] combines language-to-behavior-tree generation with command filtering and deterministic parser validation against permitted swarm primitives. This directly addresses the boundary between generated structure and accepted commands. Sangchai et al. [15] examine symbolic wrappers using structured tools or PDDL, with human review before execution. Their design makes the intermediate plan available for inspection and correction. Such mechanisms already establish validation and intermediate representations as part of language-driven robot systems; neither is a novelty claim of this paper.

Our study follows these architectural choices through execution in one implemented fleet. We examine which malformed plans are rejected, which assignments or destinations are rewritten, and which failures remain after those checks. The formation case shows how an accepted plan can still fail because of route geometry, while the completion cases distinguish component reports from the mission evaluator's ordered spatial conditions. This analysis characterizes the coverage and limits of the implemented checks. It does not compare our system's reliability with that of the approaches reviewed above.

3 System Overview

Figure 2 shows the contracts. Level 1 emits fleet-level actions in a typed plan tree. Level 2 maps those actions to coordinated waypoint sequences; Level 3 executes them. The model supplies neither velocity commands nor collision-resolution decisions. This separation makes the action fields and the available checks explicit, but a contract alone does not reject an invalid plan or guarantee successful execution.

The interface exposes four distinct correctness questions. Structural validity concerns admissible node types and fields. Task meaning concerns the assignments and goals implied by the operator's command. Geometric feasibility concerns whether robots or formations can traverse the required space. Evaluator completion concerns

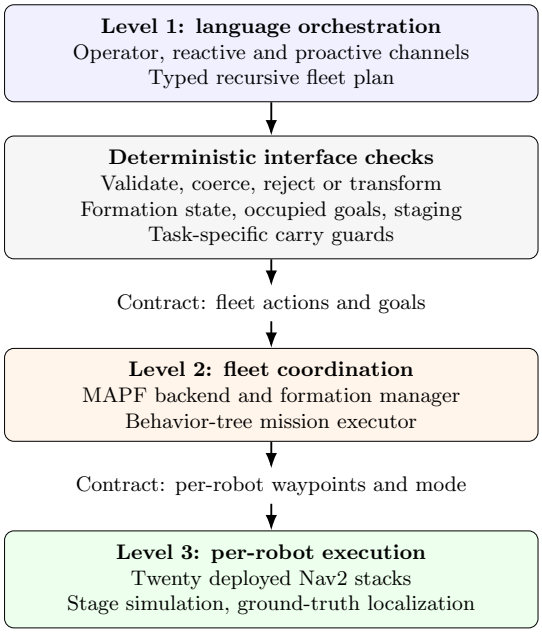


Fig. 2 Three levels and the contract at each boundary. The language model produces only the object crossing the first contract; velocities, conflict resolution, and tracking live strictly below it.

whether the specified pose-based events occur. No implication between these properties follows from the typed representation alone. Table 1 describes the implemented coverage and its limits; it is not a formal verification of system safety.

One instantiation

The following implementation supplies the setting for the case study. Twenty differential-drive robots operate in Stage [16] on a $66.6 \times 37.75\,\mathrm{m}$ non-convex map (Fig. 3) with narrow corridors and looping paths, in five groups of four with a home location each. Twenty is the deployed population, not a measured concurrency level. The reported metrics do not provide per-run counts of simultaneously moving or jointly planned robots. We therefore make no throughput, scaling, or twenty-active-agent claim.

4 Level 3: Per-Robot Execution

Each robot runs a Nav2 [17] stack with namespace substitution and staggered bring-up. The development record reports discovery and launch difficulties when starting twenty stacks simultaneously. Stage 2D was selected after Gazebo Harmonic did not provide a usable real-time factor in the development setup. These are engineering observations, not controlled simulator or DDS comparisons.

Localization uses simulator-derived poses instead of AMCL. During development, scans from nearby robots caused difficulties with static-map localization;

Table 1 Implemented check coverage and the evidence available in this study. Implemented does not mean experimentally isolated or generally sufficient.

Condition	Coverage	Evidence or limitation
Plan types and fields	Structural	Schema and recursive validators; E1 measures case-specific validity, not complete task semantics.
Robot assignments	Partial	Active-formation membership and single-carrier rules; general assignment correctness is not established.
Occupied goal cells	Local rewrite	Occupancy-aware goal adjustment does not establish route feasibility or preservation of the requested location.
Shared drop-off	Task-specific	Destination spreading is logged in the reported M3 comparison; no controlled effect estimate.
Formation assembly	Staging	Preparation exists; correct assembly does not establish traversability.
Formation route	Missing	Complete formation-footprint feasibility is not checked by the interface; the reported failure exposes this gap.
Task completion	Pose-based	Ordered dwell conditions are scored separately from action flags; physical cargo transfer, collisions and operator intent are not certified.
Repair and supervision	Implemented	Main-series counters do not exercise repeated repair; effectiveness is unmeasured.

this motivated removing localization uncertainty from the case study. The experiments therefore do not test peer-aware localization or robustness to pose noise. A tuned CycloneDDS profile and a dynamic-blocker costmap plugin are part of the implementation, but their individual effects are not measured here.

5 Level 2: Fleet Coordination

MAPF as an interchangeable backend

MAPF provides joint spatial coordination behind the fleet-action interface. Both PBS [5] and LNS2 [6] sit behind one action interface: a request carries robot identifiers and one goal point per robot, and the solver publishes the resulting conflict-free paths to the robots as ordered waypoint sequences with holds rather than returning them. Nothing above Level 2 names an algorithm, so replacing the solver is a configuration change — made true by the contract of Sec. 3, not by a coincidence of our code. All runs use LNS2.

Supporting components

Formation management and mission execution use the same separation between fleet actions and per-robot execution. A centralized manager implements leader–follower control: a specification names a leader, its followers and per-follower offsets in the leader frame; while a formation is active its members leave the free pool and the group's extent transfers to an inflated leader footprint, so MAPF plans for the group

as one body. BehaviorTree.CPP [18] runs the mission tree with parameters supplied from above.

Operational meaning of task completion

The task evaluator uses robot positions rather than component success flags. A visit task requires its configured presence condition in a target zone. A carry task requires satisfying a dwell condition in its pickup zone first and then satisfying a dwell condition in its drop-off zone. A drop-off event before the qualifying pickup does not complete the task. The reported task zone radius is 1.5 m; the numerical dwell threshold is not documented in the retained manuscript-level record and is a reproducibility limitation.

Here, pickup and drop-off are logical stages of an ordered spatial task. The success criterion does not verify grasping, attaching, transferring, or physically transporting a cargo object. We retain the term *carry* as the task label and use *evaluator completion* for its measured outcome. Completion of a mission means that all its specified task conditions have been registered; it is not a separate assertion of physical transport or collision safety.

A rewritten goal is still assessed against the original task evaluator. Its successful pose-based score therefore establishes satisfaction of that evaluator's zone conditions, not equivalence to the operator's full intent. For instance, two points inside the same acceptance zone may receive the same score while representing different requested placements. This matters when interpreting destination spreading and occupied-goal rewrites.

6 Level 1: Language Orchestration

Plan grammar

The model emits one JSON object containing a natural-language reply and a plan tree. Four leaf types cover fleet actions — `mapf` (move a robot set to a goal set), `formation` (assemble and hold), `disband`, and `idle` (do nothing, with a reason, also the escalation channel) — and two containers, `sequence` and `parallel`, nest recursively:

```
{"type":"sequence","steps":[
 {"type":"mapf","robot_ids":[8,9,10,11],
  "goals":[[-8.7,-5.9],[-9.6,-5.9],
           [-8.7,-6.8],[-9.6,-6.8]]},
 {"type":"formation",
  "leader_ns":"robot_8",
  "follower_ns":["robot_9",
   "robot_10","robot_11"],
  "offsets_x":[-1.5,-3.0,-4.5],
  "offsets_y":[0,0,0]}]}
```

The finite set of node types supports recursive validation and a JSON schema. These checks cover the implemented structural rules, not arbitrary task semantics or geometric feasibility.

Three channels and grounding

A reactive channel is invoked by behavior-tree nodes on WARN/ERROR, a proactive channel may intervene on its own initiative, and an operator chat channel streams a reply with an executable plan; all three produce objects in the same grammar and pass the same guards. Grounding facilities include a live pose cache, a room-occupancy-aware placement solver, and eight allowlisted semantic tools. Their presence does not eliminate coordinate errors; their use depends on the mission, and the main-series runs make no auxiliary lookup calls.

Generation and runtime checks

The interface combines a generation configuration with runtime checks. Schema-constrained decoding restricts the output representation at generation time; it is not a post-generation repair. Runtime components validate the schema, coerce selected field types, reject actions addressing active formation members, rewrite occupied goals, and stage robots before formation assembly. The campaign additionally introduced a single-carrier guard and a shared-drop-off guard, examined in Section 8.1. These runtime checks do not require an additional model call.

Rejection and transformation have different semantics. Rejecting an action prevents its execution under the checked condition. Transforming it changes an assignment or goal and can change the operator's intended behavior. The single-carrier rule keeps the carrier also routed to the destination; the shared-drop-off rule spreads coincident destinations. Neither is a general proof that the transformed plan preserves intent. Task completion must still be checked independently, and the implemented rules do not cover every possible conflict.

Schema restrictions also need empirical assessment. On the offline benchmark, the local 9B model's structural pass rate drops from 87.3% with native tool calling to 19.1% with schema-constrained decoding. The corresponding hosted comparison is 90.5% versus 51.4%. These are configuration-specific observations, not evidence that schema constraints generally harm planning. The simulation campaign uses the native tool-calling response configuration; this does not imply that auxiliary state-lookup tools are invoked on every mission.

Closed loop

Post-execution verification is deterministic, computed from formation status and pose state, and feeds three nested repair loops — remediation, verification repair, mission supervision — each with a step, time and no-progress budget. These mechanisms are described as implemented capabilities; the main-series counters do not establish their effectiveness.

7 Experimental Design

We report two complementary evaluations of the implemented system. The offline benchmark characterizes generated plans; the simulation campaign examines selected execution outcomes. Neither is a component-removal ablation.

E1 — structural plan accuracy, hermetic

A 44-case benchmark spanning twelve structural categories (`mapf_{basic,spread,explicit}`, `parallel`, `sequence`, `formation_{create,move,complex}`, `disband`, `idle`, `escalate`, `edge`) is run against five locally-hosted models with N=5 repeats per case at temperature 0.1: 220 calls per model, 1,100 in total. Each response is scored by programmatic validators against the structural properties the case requires, not by string match against a reference plan. The harness is hermetic: no robots run.

E2 — task-evaluator completion in simulation

The main-series report describes four cells and 18 included runs on the deployed twenty-robot population. M2 has five tasks, one labelled carry. M3 has nine tasks, three labelled carry, with two naming the same drop-off cell. The cells are a scripted M2 driver with hard-coded allocation and no model ($n = 5$), a 9B model on M2 ($n = 5$), a hosted 397B model B on M2 ($n = 5$), and B-M3 ($n = 3$). The scripted driver uses the same coordination and per-robot execution levels. There is no scripted M3 baseline, and the cells do not isolate model, provider, allocation, or guard-version effects.

The reporting record accounts for 21 sessions, 18 included and three excluded as non-valid repeats of the formation mission outside these cells. Its inclusion rules require the start configuration to be reached, one command delivery, and a finalized bag with the target topics present. It reports no outcome-based exclusion. A complete session-ID inventory and version mapping do not accompany the manuscript, so that accounting cannot be independently audited from the supplied reporting material.

Table 2 separates the evidence sources. The M3 development comparison describes two pre-guard and three post-guard runs. The post-guard runs are the three B-M3 entries in the main series; the pre-guard runs are reported as their own account and are not added to the 18-run denominator. The formation observations and qualitative defect reports are likewise not added to its denominator. Consequently, 18 successful included runs is not an all-campaign success rate.

Configuration and reproducibility

E1 reports the model identifiers, five attempts per case, temperature 0.1, and a single 16 GB GPU for local serving. These identifiers do not uniquely specify quantization, model revision, serving-library version, prompt or schema hash, endpoint configuration, or random seed. Exact per-run versions, the full prompts, and an executable offline harness are not included in the manuscript package. The numbers below are retained descriptive measurements; the present package does not support their independent recomputation.

8 Results

The results are organized around observed boundary failures before the aggregate scores. Assignment and geometric cases address RQ2; qualitative completion-report mismatches address RQ3; the offline results characterize RQ1. Each source retains its own provenance and denominator.

Table 2 Evidence sources and denominator limits. Counts are retained from the reporting record; rows are not disjoint sets that can be summed.

Source	Reported count	Use and unresolved provenance
Main mission series	18 runs	Four cells; three M2 cells and one M3 cell. Session/version inventory is not supplied.
Decision journals	13 of 13	Every language run retains a plan and a journal; eight also retain per-phase checkpoints, so execution order is inspectable only for the later cells.
Bring-up exclusions	3 sessions	Three M4 sessions excluded from the main-series denominator: pilot stack failure, leader ejection, interactive session. Two of the three are reported separately as qualitative cases; no independent ID audit.
M3 guard comparison	2 before, 3 after	Pre-guard runs are separate from the main series; the three post-guard runs are its B-M3 entries. Provider and software versions differ between series.
Formation cases	One detailed failure	Additional qualitative observations, not a systematic formation benchmark or main-series denominator.
Substrate defects	Not counted	Development observations of misleading local completion reports; no estimated frequency.

8.1 Assignment defects and shared-destination conflicts

The inspectable plans reveal an assignment-level defect relevant to RQ2. All 13 language runs retain a plan and a decision journal, but only eight also retain per-phase checkpoints; the five model-9b runs predate checkpointing and store a single record, so execution order is inspectable only for the later cells. Among the 13 plans, 5 carry the same assignment defect: the plan assigns two robots to carry a single item. In three model-9b runs the extra carrier is visible in the stored plan; in two cell-B runs the single-carrier guard had already removed it, and the defect is recorded only by the guard counter. An extra robot sent to the pickup can occupy space needed by the intended carrier. The count describes this inspected set, not the prevalence of defects across unseen missions.

Duplicate-carrier assignments

In the model-9b cell, the defect occurs in three runs, all of which complete M2. In cell B, the single-carrier guard fires in two runs, removing the extra carrier and retaining the robot routed to the drop-off. These records show that the guard handles the targeted assignment pattern. They do not show that it was necessary for task completion: the three 9B cases succeed without that correction. Execution order and corridor occupancy can affect whether an extra carrier obstructs another robot.

Shared-drop-off execution conflict

M3 explicitly asks two carry tasks to use the same drop-off location (Figure 3). Coincident destinations are therefore a coordination problem under the given mission, not

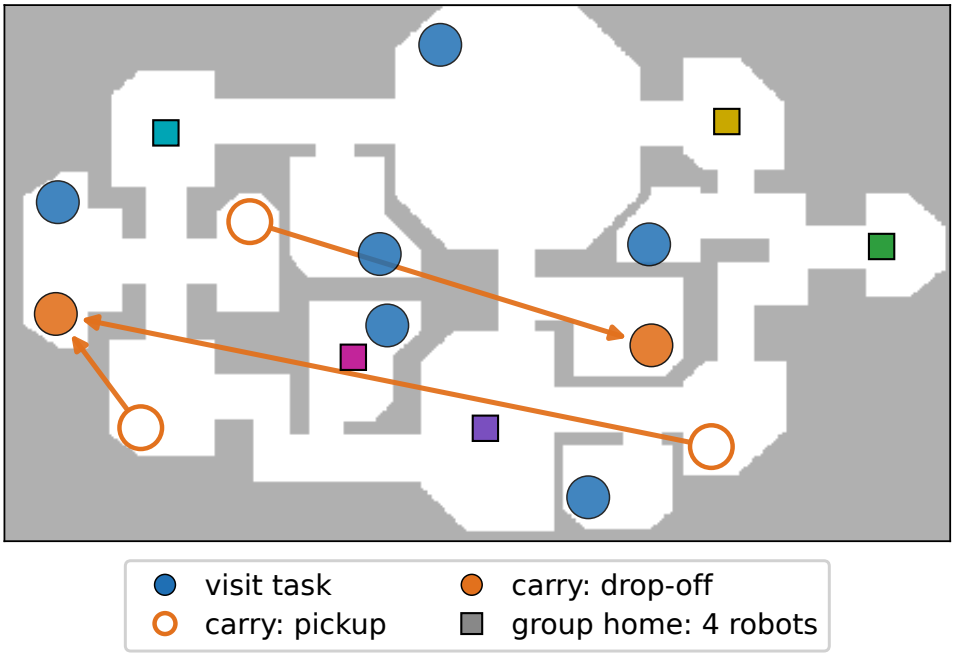


Fig. 3 The 20-robot environment (66.6 × 37.75 m, 0.05 m cells) with the nine task zones of mission M3 drawn at their true 1.5 m radius. Blue: visit tasks. Orange: the three carry tasks, pickup to drop-off. Two of the three name the same drop-off cell, creating an execution conflict handled by the shared-drop-off guard (Sec. 8.1). Mission M2 uses the five-task subset.

by themselves evidence that the model misinterpreted the request. Before the shared-drop-off guard, two consecutive development runs complete 8/9 tasks, both on the same provider. Three subsequent runs with the guard complete 9/9 tasks, and their logs record destination spreading:

```
carry_guard: carry_engine_to_reactor+
carry_shields_to_reactor share dropoff
[-29.65,-2.85], spread to robot_3, robot_16
```

This is a small before/after case study, not a randomized or paired ablation. The observations are consistent with the intended conflict-handling mechanism; changing providers and software versions prevents attributing the entire completion difference to that guard alone.

The two carry cases support different conclusions. A duplicate-carrier assignment can coexist with successful execution, and its correction is visible in logs. A mission-specified shared destination can require spatial adjustment even when the requested plan is otherwise appropriate. Combining these cases into one defect count or one general guard-benefit estimate would lose that distinction.

8.2 Case study: a formation that cannot move

One run is worth reporting on its own, with n=1 stated plainly. Asked for a column, the model emitted `offsets_x=[0,0,0]`, `offsets_y=[1.5,-1.5,3.0]` — geometrically a line abreast 4.5 m wide, not a column. The formation itself assembled and held, stable in 6.5 s and never broken — though under static localization the tracking error these runs report is that of the controller given an exact pose estimate, and should not be read as a formation-control result. It then could not move at all:

```
robot_0: no static path ... (footprint won't fit through)
```

With a formation active the group's extent transfers to an inflated leader footprint, and the passages this map offers are narrower than that extent.

The plan is structurally valid and passes the implemented interface checks, yet mismatches the requested column and cannot traverse the corridors of this map while keeping that formation: the occupancy-aware goal rewrite does consult the environment, but the implemented checks do not establish traversability of the complete formation footprint along the route. This motivates an additional check — formation feasibility against the occupancy grid — which we do not implement, and which the contract of Sec. 3 places at Level 2, the lowest level holding both the group extent and the map. A separate observation with the same command produced appropriate offsets. These cases show output variability, but do not estimate a formation failure rate or establish systematic behavior.

8.3 Local action reports and mission conditions

This is a qualitative finding and we label it as such: the defects below were found and repaired as they appeared while building the system, not counted under a protocol, so we give no rates. It identifies a recurring engineering concern in this implementation: components can report local success without satisfying the mission-level completion condition. A misleading completion report can affect later execution and any repair decision based on that report. This is distinct from a detectable plan-schema violation.

The pattern recurred in several forms. An executor step for a task labelled carry reported all agents arrived while one robot had never left its spawn. A robot benched by a guard was silently dropped from the requested set and never reassigned. A blocked-goal rewrite moved a target by two metres and the robot reported arrival somewhere the mission had not asked for. A formation manager refused a request immediately while the executor learned of it only at step-budget expiry. These reports did not by themselves establish that every requested agent or original goal had satisfied the task conditions. The examples are not a count of additional failed experimental runs.

Some reports reflect a component's reduced local request rather than the original mission requirement. Others arise when layers exchange incomplete status information. Component tests alone may miss such interactions; the reported examples motivate explicit completion conditions and integration checks, but were not collected as a quantified failure study.

Hence the choice we adopted: score missions using observed pose state rather than treating an action-status flag as sufficient evidence of task completion. A contract should also specify what counts as completion at its boundary, not only what crosses it.

8.4 Structural plan errors and serving configurations

Table 3 reports E1. Local structural pass rates range from 70.0% to 87.3%. The 9B model has the highest observed local score, 8.7 percentage points above the 24B model and 12.3 above the 14B model. Model size therefore does not order these particular configurations. The comparison does not isolate parameter count from model family, quantization, or serving behavior.

Five repeats characterize response variability on the same 44 cases; they are not 220 independent task designs. We report descriptive pass rates and the fraction of cases with at least one valid response among five attempts, without model-ranking significance claims or confidence intervals that treat repeated cases as independent. For the 4B model, the difference between pass@1 and pass@5 is 23.2 points; other models recover less through resampling. The same offsets-arity error accounts for 20 of the 24B model's 47 failures, and the most frequent response is byte-identical in 8 of them. This is a repeated error pattern, not evidence that generation is deterministic.

Offline rejection rates cannot be read as mission failure rates. The validators assess case-specific structural properties without a running fleet; runtime transformations and execution ordering can change outcomes.

Table 3 Structural plan accuracy on the 44-case benchmark, N=5, 220 calls per model per configuration. *tools* is native tool calling, the deployed configuration; *schema* is schema-constrained decoding. pass@1 is call-level (passed calls over total calls); pass@5 is case-level (cases with at least one valid response in five attempts). Two hosted models are mixture-of-experts, reported as total/active parameters; gemini does not disclose its size.

Model	params tot./act.	pass@1 % tools	pass@5 tools	pass@1 schema
local, single 16 GB card, all dense				
qwen3.5:4b	4B	70.0	93.2	—
gemma3:12b	12B	71.4	72.7	—
qwen2.5:14b	14B	75.0	81.8	—
qwen3.5-9b	9B	**87.3**	**95.5**	19.1
mistral-small3.2:24b	24B	78.6	81.8	—
hosted				
qwen3.5-9b	9B dense	90.5	97.7	51.4
gemini-2.5-flash	n/d	89.5	90.9	80.9
nemotron-3-ultra	550B/55B	90.5	97.7	88.6
qwen3.5-397b-a17b	397B/17B	**93.2**	**100**	**95.0**

8.4.1 Hosted and constrained-decoding observations

The hosted block applies the same benchmark to four models. Two are mixture-of-experts, so total and active parameter counts are shown separately. These results characterize the tested configurations; they do not identify the causal effect of model scale.

Schema-constrained decoding changes the scores unevenly. The hosted 397B/17B model rises from 93.2% to 95.0%, while Gemini falls from 89.5% to 80.9%. For the 9B model, the decrease is 39.1 points remotely and 68.2 locally. Without a controlled serving-stack experiment, these differences cannot be attributed to one decoding mechanism. They motivate retaining representation validity and task-plan validity as separate evaluation targets. Investigation of the model-specific causes is outside this study.

8.5 Main-series task-evaluator completion

Table 4 gives E2. Every reported cell satisfies all assigned task-evaluator conditions. The reported series therefore has no task-completion variation with which to estimate a language-model or guard effect. Timing differs descriptively, as discussed below.

Table 4 E2: 18 included runs with twenty deployed robots. Tasks are pose-based evaluator outcomes, not physical cargo-transfer measurements. *in model* is wall-clock time spent waiting on the language model. Dispersion is the sample standard deviation ($n - 1$).

Cell	Mission	n	Tasks	makespan (s) mean ± sd	in model (s)
no-LLM	M2	5	5/5	203.5 ± 10.6	0
model-9b	M2	5	5/5	252.2 ± 26.6	9.5
B	M2	5	5/5	260.0 ± 16.5	31.6
B-M3	M3	3	9/9	587.7 ± 67.0	51.2

All reported main-series runs complete their tasks, but the missions cover one map and two task specifications. This observation establishes execution on those cases, not equivalence between the arms or reliability on unseen missions. The separate pre-guard and formation cases also show why this completion column is not a summary of every development run.

8.6 Timing and inactive mechanisms

The M2 language cells have reported mean makespans of 252.2 ± 26.6 s for 9B and 260.0 ± 16.5 s for B. Both complete all tasks, but they differ in model, provider, allocation, and guard version. These measurements do not establish an effect or absence of effect of model size.

The B mean exceeds the scripted baseline by 56.5 s (203.5 versus 260.0 s), with 31.6 s recorded as model waiting time. Makespan is measured from the first

`/tasks/state` tick in the bag, not from command submission. The recording-to-command offset was not measured consistently, and allocations differ. Thus the gap is a descriptive comparison of recorded intervals, not a controlled language-interface overhead or operator-to-completion latency estimate.

Across the 13 language runs, remediation attempts, verification-repair attempts, and active-formation-guard activations are all zero; supervision records one pass rather than a repeated loop. M2 and M3 therefore do not evaluate those mechanisms. Auxiliary state-lookup tool calls are also zero in these runs: named-room coordinates are already supplied by the static prompt context. Separate formation observations record 10 and 33 calls, but are not part of the main M2/M3 comparison. This distinction separates the native tool-calling output configuration from actual use of grounding tools.

9 Discussion

The results show why structural validity is a useful first check but an incomplete account of fleet behavior. For RQ1, the benchmark distinguishes tested structural rules from mission execution. The repeated malformed offset array is an observed error pattern, while the tool/schema comparisons expose configuration sensitivity. Aggregate pass rates and the reported offset example do not constitute a complete per-category error taxonomy; the retained reporting record does not provide the underlying responses needed to reconstruct one.

Passing those structural checks leaves the assignment and spatial questions of RQ2. The single-carrier cases show targeted assignment correction, with successful uncorrected cases preventing a necessity claim. The shared-drop-off account shows destination spreading consistent with the intended coordination rule, without isolating its causal effect. The formation case reveals both a semantic mismatch and a missing footprint-route check. Together these observations identify different checks for different properties. A parseable plan, a locally admissible goal, and a traversable group route are distinct requirements.

The distinction extends to execution feedback, as examined in RQ3. A component may report success for an adjusted goal or reduced robot set without satisfying the original mission condition. The pose-based evaluator provides a separate check of the specified spatial events, but its own specification is also limited: ordered dwell success does not establish physical transport, intent preservation, or collision safety. An engineering implication is to record the requested action, its transformed action, the component report, and the evaluator outcome separately. This is a lesson drawn from the cases, not a demonstrated general reliability improvement.

Statements and Declarations

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Funding

[Funding statement to be confirmed with the project supervisor before submission. The work was carried out as part of the authors' employment at the Institute of Robotics and Computer Vision, Innopolis University; whether it constitutes external funding is for the institution to state.]

Ethics Approval

Not applicable. This study did not involve human participants or animals.

Consent to Participate

Not applicable.

Consent for Publication

Not applicable.

Data Availability

The simulation code, the eighteen included runs with their per-run metadata, decision journals, derived metrics and analysis scripts are available in the project repository at https://github.com/Innopolis-Robotics-Society/llm_swarm, on the default branch. The recorded rosbag payloads (approximately 2 GB) are not tracked in the repository; they are available from the corresponding author on reasonable request. No new simulation runs or offline model calls were performed for this revision; the reported values are retained from the existing reporting record.

Code Availability

The ROS 2 simulation stack, the MAPF planners, the language-orchestration components and the analysis scripts are in the same repository, on the default branch. The repository carries no separate licence file; the code is made available for review and for reproduction of the reported results.

Author Contributions

A.T.: Conceptualization, Methodology, Software, Investigation, Data curation, Writing—original draft, Writing—review and editing, Project administration. D.V.: Software, Investigation, Writing—review and editing. E.S.: Conceptualization, Methodology, Software, Investigation, Supervision, Project administration, Writing—review and editing. S.B.: Software, Investigation, Visualization. A.Mal.: Software, Validation. A.Malol.: Supervision, Conceptualization, Writing—review and editing.

10 Limitations

The study uses one Stage map, simulator-derived poses, two main mission specifications, and small reported samples. Twenty deployed stacks do not establish twenty simultaneously active or jointly planned agents. No per-run activity trace, fleet-size sweep, or throughput analysis is supplied. The carry metric is ordered zone dwell; it does not test physical load handling. There are no independent collision or intent-preservation metrics. These limits preclude fleet-wide reliability, physical transport, and safety claims.

The main-series completion metric is saturated. Five runs store a single-record journal without per-phase checkpoints, biasing inspection of execution order toward the later cells. The M3 before/after account is identified by session, but its two series differ in provider and software version. The absence of a complete ID/version registry limits auditability and cannot be repaired merely by naming the series separately. The reported 18-run result must not be interpreted as an all-campaign success probability.

E1 repeats 44 case designs and measures selected structural requirements. It does not estimate general language competence or end-to-end success. The reporting package omits exact prompts, revisions, quantization settings, serving configurations and seeds required for full reproduction. Models and serving settings are confounded. Makespan is anchored to the first task-state tick, with uncontrolled command offset and allocation, so it does not isolate language overhead or operator latency.

No controlled component-removal ablation was performed. The six-case drop-off account changes providers and software versions, so a causal guard-benefit estimate is unavailable. Inactive repair counters do not validate recovery behavior. Goal and assignment rewrites can change task meaning, and formation-route feasibility remains unchecked. Qualitative substrate observations motivate checks but establish no failure frequencies. These are limitations of the evidence, not deficiencies that stronger wording or architectural integration can remove.

11 Conclusion

This architectural case study distinguishes the correctness properties at the boundary between language plans and classical fleet execution. The reported offline cases expose structural errors and configuration sensitivity. Carry-labelled simulation cases demonstrate selected assignment and goal transformations, while the formation and local-status observations expose conditions that the implemented interface does not establish.

The central result is a documented separation of structural validity, task meaning, spatial feasibility, and pose-based evaluator completion in this implementation. The main-series task outcomes apply to the reported missions and their ordered dwell specification. They do not prove improved guard reliability, physical cargo transport, collision safety, or twenty-agent concurrent scaling. Better artifact traceability and controlled evaluation would be needed to establish those stronger claims.

References

[1] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., *et al.*: Do as I can, not as I say: Grounding language in robotic affordances. In: Proceedings of the 6th Conference on Robot Learning (CoRL). Proceedings of Machine Learning Research, vol. 205, pp. 287–318 (2022)

[2] Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., Zeng, A.: Code as policies: Language model programs for embodied control. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 9493–9500 (2023). https://doi.org/10.1109/ICRA48891.2023.10160591

[3] Chen, W., Koenig, S., Dilkina, B.: Why Solving Multi-agent Path Finding with Large Language Model has not Succeeded Yet. arXiv:2401.03630 (2024). https://doi.org/10.48550/arXiv.2401.03630

[4] Sharon, G., Stern, R., Felner, A., Sturtevant, N.R.: Conflict-based search for optimal multi-agent pathfinding. Artificial Intelligence **219**, 40–66 (2015) https://doi.org/10.1016/j.artint.2014.11.006

[5] Ma, H., Harabor, D., Stuckey, P.J., Li, J., Koenig, S.: Searching with consistent prioritization for multi-agent path finding. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, pp. 7643–7650 (2019). https://doi.org/10.1609/aaai.v33i01.33017643

[6] Li, J., Chen, Z., Harabor, D., Stuckey, P.J., Koenig, S.: MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, pp. 10256–10265 (2022). https://doi.org/10.1609/aaai.v36i9.21266

[7] Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., Sermanet, P., Brown, N., Jackson, T., Luu, L., Levine, S., Hausman, K., Ichter, B.: Inner monologue: Embodied reasoning through planning with language models. In: Proceedings of the 6th Conference on Robot Learning (CoRL). Proceedings of Machine Learning Research, vol. 205, pp. 1769–1782 (2022)

[8] Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., Garg, A.: ProgPrompt: Generating situated robot task plans using large language models. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 11523–11530 (2023). https://doi.org/10.1109/ICRA48891.2023.10161317

[9] Li, P., An, Z., Abrar, S., Zhou, L.: Large language models for multi-robot systems: a survey. Autonomous Robots **50**, 30 (2026) https://doi.org/10.1007/s10514-026-10257-4

[10] Kannan, S.S., Venkatesh, V.L.N., Min, B.-C.: SMART-LLM: Smart Multi-Agent Robot Task Planning using Large Language Models. arXiv:2309.10062, version 2 (2024). https://doi.org/10.48550/arXiv.2309.10062

[11] Mandi, Z., Jain, S., Song, S.: RoCo: Dialectic multi-robot collaboration with large language models. In: 2024 IEEE International Conference on Robotics and Automation (ICRA), pp. 286–299 (2024). https://doi.org/10.1109/ICRA57147.2024.10611491

[12] Zhang, X., Qin, H., Wang, F., Dong, Y., Li, J.: LaMMA-P: Generalizable multi-agent long-horizon task allocation and planning with LM-driven PDDL planner. In: 2025 IEEE International Conference on Robotics and Automation (ICRA) (2025). IEEE. https://arxiv.org/abs/2409.20560

[13] Tran, D., Arefin, M., Ghosh, B., Rahmani, A., He, H.: HiP-CL: High-level planning based on common language for multi-robot teaming. Human-Centric Intelligent Systems **6**, 93–106 (2026) https://doi.org/10.1007/s44230-025-00130-5

[14] Majid, M., Majid, A.Y.: CommandSwarm: Safety-Aware Natural Language-to-Behavior-Tree Generation for Robotic Swarms. arXiv:2605.07764 (2026). https://doi.org/10.48550/arXiv.2605.07764

[15] Sangchai, K., Boonpun, M., Kraipetchara, W., Garcia, P.: Symbolic wrappers enable precise human in the loop operation for large action models. Discover Robotics **2**, 8 (2026) https://doi.org/10.1007/s44430-026-00023-7

[16] Vaughan, R.: Massively multi-robot simulation in stage. Swarm Intelligence **2**(2–4), 189–208 (2008) https://doi.org/10.1007/s11721-008-0014-4

[17] Macenski, S., Martín, F., White, R., Clavero, J.G.: The marathon 2: A navigation system. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 2718–2725 (2020). https://doi.org/10.1109/IROS45743.2020.9341207

[18] Faconti, D.: BehaviorTree.CPP. Open-source library, accessed 2026 (2024). https://www.behaviortree.dev